

Neural Engine: Deep Technical Analysis

Orel Shushan - Software Engineering

May 2026

Contents

1	1. Project System Overview	1
2	2. Key Engineering Principles	1
3	3. Deep Component Analysis	2
4	Architecture: Layer Component	2
4.1	Functional Decomposition	2
4.1.1	Method: Layer	2
4.1.2	Method: forward	2
4.1.3	Method: backward	2
5	Architecture: Matrix Component	2
5.1	Functional Decomposition	2
5.1.1	Method: Matrix	2
5.1.2	Method: Matrix	2
5.1.3	Method: Matrix	2
5.1.4	Method: operator	2
5.1.5	Method: randomize	3
5.1.6	Method: fillZero	3
5.1.7	Method: applyReLU	3
6	Architecture: Network Component	3
6.1	Functional Decomposition	3
6.1.1	Method: Network	3
6.1.2	Method: feedForward	3
6.1.3	Method: backpropagate	3
6.1.4	Method: train	3

1 1. Project System Overview

This report provides a formal decomposition of the Neural Network Skeleton. The engine is designed for high-performance matrix operations and gradient-based learning.

2 2. Key Engineering Principles

- **Cache Coherency:** Matrix operations are optimized for sequential memory access.
- **Move Semantics:** Leveraging C++14 to eliminate redundant memory allocations.
- **Modular Design:** Separation of Linear Algebra (Matrix) and Neural Logic (Layer/Network).

3 3. Deep Component Analysis

4 Architecture: Layer Component

The `Layer` class is a fundamental building block of the neural engine found in `Layer.h`.

4.1 Functional Decomposition

4.1.1 Method: `Layer`

Signature: `public: Layer(int inputSize, int outputSize)`

Engineering Context: Core internal logic for neural computations.

4.1.2 Method: `forward`

Signature: `Matrix forward(const Matrix& input)`

Engineering Context: Core internal logic for neural computations.

4.1.3 Method: `backward`

Signature: `Matrix backward(const Matrix& outputGradient, double learningRate)`

Engineering Context: Calculates the local gradient for a specific layer and updates parameters using Stochastic Gradient Descent.

5 Architecture: Matrix Component

The `Matrix` class is a fundamental building block of the neural engine found in `Matrix.h`.

5.1 Functional Decomposition

5.1.1 Method: `Matrix`

Signature: `public: Matrix(int r, int c)`

Engineering Context: Core internal logic for neural computations.

5.1.2 Method: `Matrix`

Signature: `Matrix(const Matrix& other)`

Engineering Context: Core internal logic for neural computations.

5.1.3 Method: `Matrix`

Signature: `Matrix()`

Engineering Context: Core internal logic for neural computations.

5.1.4 Method: `operator`

Signature: `double& operator()(int row, int col)`

Engineering Context: Core internal logic for neural computations.

5.1.5 Method: `randomize`

Signature: `void randomize(double min, double max)`

Engineering Context: Breaks symmetry by initializing weights with small random values, allowing neurons to learn distinct features.

5.1.6 Method: `fillZero`

Signature: `void fillZero()`

Engineering Context: Core internal logic for neural computations.

5.1.7 Method: `applyReLU`

Signature: `void applyReLU()`

Engineering Context: Applies the non-linear activation function $\max(0, x)$ to introduce complex decision boundaries.

6 Architecture: Network Component

The `Network` class is a fundamental building block of the neural engine found in `Network.h`.

6.1 Functional Decomposition

6.1.1 Method: `Network`

Signature: `public: Network(int* layerSizes, int layerSize, double lr)`

Engineering Context: Core internal logic for neural computations.

6.1.2 Method: `feedForward`

Signature: `Matrix feedForward(const Matrix& input)`

Engineering Context: Propagates the input signal through the network layers using the formula $Z = W \cdot A + B$.

6.1.3 Method: `backpropagate`

Signature: `void backpropagate(const Matrix& input, const Matrix& target)`

Engineering Context: Calculates gradients using the Chain Rule to minimize the loss function.

6.1.4 Method: `train`

Signature: `void train(Matrix* inputs, Matrix* targets, int numSamples, int epochs)`

Engineering Context: The primary optimization loop. It iterates over the dataset for multiple epochs to refine the model's accuracy.